

```
In [13]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [14]: df = sns.load_dataset("titanic")
df.head()
```

Out[14]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton

```
In [15]: # Display dataset information
print(df.info())

# Display statistical summary
print(df.describe())

# Display missing values
print(df.isnull().sum())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    str
3   age         714 non-null    float64
4   sibsp       891 non-null    int64
5   parch       891 non-null    int64
6   fare        891 non-null    float64
7   embarked    889 non-null    str
8   class       891 non-null    category
9   who         891 non-null    str
10  adult_male  891 non-null    bool
11  deck        203 non-null    category
12  embark_town 889 non-null    str
13  alive       891 non-null    str
14  alone       891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), str(5)
memory usage: 80.7 KB
None
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

```
In [16]: # Fill missing values in age with median
```

```

df["age"] = df["age"].fillna(df["age"].median())

# Fill missing values in embark_town with mode
df["embark_town"] = df["embark_town"].fillna(df["embark_town"].mode()[0])

# Remove deck column because it has many missing values
df.drop(columns=["deck"], inplace=True)

# Check missing values again
print(df.isnull().sum())

```

```

survived      0
pclass        0
sex            0
age            0
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
embark_town   0
alive         0
alone         0
dtype: int64

```

```

In [17]: # Create LabelEncoder object
le = LabelEncoder()

# Convert sex column
df["sex"] = le.fit_transform(df["sex"])

# Convert embark_town column
df["embark_town"] = le.fit_transform(df["embark_town"])

# Remove unnecessary columns
df.drop(columns=[
    "embarked",
    "class",
    "who",
    "alive",
    "adult_male",
    "alone"
], inplace=True)

# Display first 5 rows
df.head()

```

```

Out[17]:
   survived  pclass  sex  age  sibsp  parch  fare  embark_town
0         0       3    1  22.0     1     0  7.2500             2
1         1       1    0  38.0     1     0 71.2833             0
2         1       3    0  26.0     0     0  7.9250             2
3         1       1    0  35.0     1     0 53.1000             2
4         0       3    1  35.0     0     0  8.0500             2

```

```

In [18]: # Create StandardScaler object
scaler = StandardScaler()

# Numerical columns to scale
numerical_cols = ["age", "fare"]

# Scale the numerical columns
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

# Display first 5 rows
df.head()

```

```

Out[18]:
   survived  pclass  sex  age  sibsp  parch  fare  embark_town
0         0       3    1 -0.565736     1     0 -0.502445             2
1         1       1    0  0.663861     1     0  0.786845             0
2         1       3    0 -0.258337     0     0 -0.488854             2
3         1       1    0  0.433312     1     0  0.420730             2
4         0       3    1  0.433312     0     0 -0.486337             2

```

```
In [19]: # Features
X = df.drop("survived", axis=1)

# Target
y = df["survived"]

# Split dataset into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
In [20]: print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

print("\nFirst 5 Rows of Training Data:")

X_train.head()
```

Training Data Shape: (712, 7)
Testing Data Shape: (179, 7)

First 5 Rows of Training Data:

```
Out[20]:
```

	pclass	sex	age	sibsp	parch	fare	embark_town
331	1	1	1.240235	0	0	-0.074583	2
733	2	1	-0.488887	0	0	-0.386671	2
382	3	1	0.202762	0	0	-0.488854	2
704	3	1	-0.258337	1	0	-0.490280	2
813	3	0	-1.795334	4	2	-0.018709	2

```
In [21]: # Save the cleaned dataset
df.to_csv("preprocessed_titanic.csv", index=False)

print("Dataset saved successfully!")
```

Dataset saved successfully!

```
In [ ]:
```